

Class-1

1. Reverse

Reverse a part of an array **in place**: two pointers, swap, move inward.

```
python

def reverse(arr, lo, hi):
    while lo < hi:
        arr[lo], arr[hi] = arr[hi], arr[lo]
        lo, hi = lo + 1, hi - 1
```

Examples:

```
reverse([a b c d e], 0, 4)  ->  e d c b a      (whole array)
reverse([a b c d e], 1, 3)  ->  a d c b e      (middle only)
reverse([a b c d e], 2, 2)  ->  a b c d e      (single element: no change)
```

Uses no extra memory. This tiny function is a **building block** — remember it.

2. Rotate

Rotate left by k = move the first k elements to the back.

```
rotate_left([a b c d e f], k=2)  ->  c d e f a b
```

Trick: **three reverses**.

```
python

def rotate_left(arr, length, k):
    k %= length
    reverse(arr, 0, k - 1)           # reverse first k
    reverse(arr, k, length - 1)      # reverse the rest
    reverse(arr, 0, length - 1)      # reverse everything
```

Dry run, k = 2:

```
a b c d e f
b a | c d e f      reverse [0..1]
b a | f e d c      reverse [2..5]
c d e f a b        reverse [0..5]   done!
```

Why it works: rotation swaps two blocks ($XY \rightarrow YX$), and ($YX = \text{reverse}(\text{reverse}(X) + \text{reverse}(Y))$). Verify on paper with $X = ab, Y = cdef$.

3. shift_left

Slide a block of the array to the left by `gap` positions (used to close holes).

python

```
def shift_left(arr, lo, hi, gap):    # move arr[lo:hi] left by gap
    for i in range(lo, hi):
        arr[i - gap] = arr[i]
```

Dry run: `shift_left(arr, 4, 8, 2)` — move `arr[4..7]` left by 2:

idx:	0 1 2 3 4 5 6 7	
	a b _ _ c d e f	we want c d e f to start at index 2
i=4:	a b c _ c d e f	arr[2] = arr[4]
i=5:	a b c d c d e f	arr[3] = arr[5]
i=6:	a b c d e d e f	arr[4] = arr[6]
i=7:	a b c d e f e f	arr[5] = arr[7]
result:	a b c d e f (e f)	positions 6,7 are leftover junk — ignore them

Note: we copy **left to right**. Copying right to left would overwrite data before reading it. Direction matters when regions overlap!

4. run_end

Find where a "run" (block of equal characters) ends.

python

```
def run_end(arr, i, hi):    # first index PAST the run starting at i
    j = i
    while j < hi and arr[j] == arr[i]:
        j += 1
    return j
```

Example on `a a a b b c`:

<code>run_end(arr, 0, 6) = 3</code>	(run 'aaa' is arr[0..2])
<code>run_end(arr, 3, 6) = 5</code>	(run 'bb' is arr[3..4])
<code>run_end(arr, 5, 6) = 6</code>	(run 'c' is arr[5])

Pattern to scan all runs: `i = 0; while i < n: j = run_end(arr, i, n); ...; i = j`

5. The problem + the 2-pass solution

Problem: compress runs in place: (aaa -> a3), (abcccccc -> a1b1c6). You are **given** that the answer always fits in the array.

Why it is hard: a single character GROWS: (g -> g1) (1 cell becomes 2). Naive left-to-right writing destroys unread characters.

Idea: runs of length ≥ 2 always shrink or stay equal. So:

- **Pass 1 (left to right):** compress runs ≥ 2 ; copy singles through *bare* (no count).
- **Pass 2 (right to left):** expand each bare single to (x1), writing from the back.

Dry run on (abcdddddddef):

```
idx:  0 1 2 3 4 5 6 7 8 9 0 1 2
      a b c d d d d d d e f      input
PASS 1 (forward):
      a b c d 8 e f . . . . .      a,b,c bare | d×8 -> 'd8' | e,f bare
      intermediate = "abcd8ef" (7 cells), rest is junk

final length L = 7 + (number of bare singles = 5) = 12    (fits: 12 <= 13)

PASS 2 (backward, read from index 6, write from index 11):
      a b c d 8 e . . . . f 1 .      'f' has no digit after it -> f1
      a b c d 8 . . . e 1 f 1 .      'e' -> e1
      a b c . . . d 8 e 1 f 1 .      'd8' is a finished token -> copy
      a b . . c 1 d 8 e 1 f 1 .      'c' -> c1
      a . b 1 c 1 d 8 e 1 f 1 .      'b' -> b1
      a 1 b 1 c 1 d 8 e 1 f 1 .      'a' -> a1

answer = a1b1c1d8e1f1
```

Safety in one line each: pass 1 never grows (write chases read, going right); pass 2 never shrinks (write stays ahead of read, going left).

⚠ **The catch (read this last, remember it forever):** pass 2 recognizes tokens by asking "*is the next character a digit?*" That only works because letters and digits are different! If the input itself can contain digits, look what pass 1 produces:

```
input:   a a a 3 d d 1 1 e e 4 f f 6 6 6
pass 1:  a 3 3 d 2 1 2 e 2 4 f 2 6 3
```

Which digits are counts and which are input characters? **Impossible to tell.** The 2-pass solution only works for NON-DIGIT inputs. For digits we need a new idea (section 11).

6. The benefit function

For each run, define its **benefit**:

$$E(\text{run}) = \text{run length} - \text{token length} \quad (\text{token} = \text{char} + \text{count digits})$$

run	token	token length	E
aaaaa	a5	2	+3
ee	e2	2	0
b	b1	2	-1
a × 12	a12	3	+9

python

```
def token_length(run):  
    return 1 + len(str(run))          # '1 +' for the char itself  
  
def benefit(run):  
    return run - token_length(run)
```

Meaning: E = how many cells of *free space* this run donates (+) or demands (-). "The answer always fits" translates to: **sum of all E ≥ 0**.

7. find_start

If we encode every run fully ((char+count), always) in one sweep, a running total of E must **never go below 0** — otherwise the writer overtakes unread data. Trick: we may start the sweep at any run and wrap around. Find a safe start:

python

```
def find_start(arr):  
    n = len(arr)  
    total, pos, i = 0, 0, 0  
    while i < n:  
        j = run_end(arr, i, n)  
        total += (j - i) - token_length(j - i)  
        if total < 0:          # bankrupt!  
            total = 0          # forget this failed stretch  
            pos = j            # try starting at the NEXT run  
        i = j  
    return pos
```

Example 1: (abccdddefggggghhhh)

```
run:  a   b   cc  ddd  e   f   ggggg  hhhhh
E:    -1  -1   0   +1  -1  -1   +3    +3
total: -1! -1!  0    1   0  -1!   3     6
        ^reset ^reset      ^reset -> pos = 9 (the g-run)
```

Example 2 (why the reset matters!): (abcdefffffghhhh)

```
run:  a   b   c   d   e   fffffff  g   hhhh
E:    -1  -1  -1  -1  -1     +5     -1   +2
```

Without resetting, the plain running sum is $(-1 -2 -3 -4 -5 0 -1 +1)$; its last negative moment is at (g), which wrongly suggests starting at (hhhh). But from (hhhh): +2, then wrap into five singles: 1, 0, -1 → crash! With resets: reset after a,b,c,d,e → candidate = (ffffff); then 5,4,6 — never negative → **pos = the f-run**. Check the wrap: 5,4,6,5,4,3,2,1 ✓.

8. The Gas Station problem (LeetCode 134)

<https://leetcode.com/problems/gas-station/description/>

Circular road, n stations. Station i gives $\text{gas}[i]$; driving to the next costs $\text{cost}[i]$. Tank starts empty. Find a starting station that completes the circle.

python

```
class Solution:
    def canCompleteCircuit(self, gas: List[int], cost: List[int]) -> int:
        if sum(gas) < sum(cost):
            return -1          # not enough fuel in total: impossible

        start = 0
        tank = 0
        for i in range(len(gas)):
            tank += gas[i] - cost[i]
            if tank < 0:        # bankrupt before completing
                tank = 0       # reset
                start = i + 1   # candidate = next station
        return start
```

Dry run: (gas = [1,2,3,4,5]), (cost = [3,4,5,1,2]) → net = $(-2, -2, -2, +3, +3)$

```

i=0: tank=-2 <0 -> reset, start=1
i=1: tank=-2 <0 -> reset, start=2
i=2: tank=-2 <0 -> reset, start=3
i=3: tank=+3
i=4: tank=+6 -> answer: start = 3

```

Why the greedy is correct (intuition):

- If you start at A and first go broke at B, then **no station between A and B can work either** — starting later only means arriving at B with less fuel (you gave up a non-negative head start).
- If total fuel $\geq$ total cost, the last surviving candidate must work.

9. Kadane's algorithm (Maximum Subarray, LeetCode 53)

Find the maximum sum of any contiguous subarray.

```

python
def max_subarray(arr):
    best = cur = arr[0]
    for x in arr[1:]:
        cur = max(x, cur + x)    # extend, OR restart at x
        best = max(best, cur)
    return best

```

Dry run on `[-2, 1, -3, 4, -1, 2, 1, -5, 4]`:

x:	-2	1	-3	4	-1	2	1	-5	4
cur:	-2	1	-2	4	3	5	6	1	5
best:	-2	1	1	4	4	5	6	6	6

-> answer 6 ([4, -1, 2, 1])

The key line: `cur = max(x, cur + x)`. If `cur` has gone negative, `x` alone is bigger than `cur + x` — so we **drop the old prefix and restart**. A negative prefix can never help whatever comes next.

10. The same pattern three times

	Gas Station	Kadane	find_start
running quantity	tank (fuel)	cur (sum)	total (spare cells)
add each step	gas[i] - cost[i]	arr[i]	E(run)
goes negative means	can't reach next stop	prefix hurts the future	writer would smash unread data
the reset	tank=0, start=i+1	cur = x (restart here)	total=0, pos = next run
why abandon is safe	any inner start arrives poorer	a negative prefix never helps	same as gas station

One sentence to remember:

When a running total goes negative, the whole prefix is dead weight — throw it away and restart. One pass, $O(n)$.

If a problem says "a running quantity must stay non-negative and you may choose where to begin" — this is the pattern.

11. Next level: characters can be digits

The 2-pass solution is dead (section 5). New plan — never leave the string in a half-done, unreadable state:

1. Find a safe starting run with `find_start` (sections 6-7).
2. Encode **every** run fully as `char+count`, one sweep, starting at `pos`, wrapping past the array's end to the front if needed.
3. The result may sit in the wrong place / two pieces → repair with `shift_left` and `rotate_left` (sections 2-3).

No code here — just watch it work.

Example A (easy: output stays in one piece): `abTTTTTTTTTT` ($n = 12$)

```
runs: a(-1)  b(-1)  T×10(E = 10-3 = +7)          find_start: pos = 2
idx:  0 1 2 3 4 5 6 7 8 9 0 1
      a b T T T T T T T T T T
      a b T 1 0 T T T T T T T
      a b T 1 0 a 1 T T T T T
      a b T 1 0 a 1 b 1 T T T
      T 1 0 a 1 b 1 . . . . .
      a 1 b 1 T 1 0 . . . . .
answer = a1b1T10

input
encode from pos: T×10 -> 'T10' at [2..4]
wrap to the skipped runs: a1 at [5..6]
b1 at [7..8]    - done, one piece [2..9]
shift_left by 2: slide block to front
rotate_left by 3 (= size of 'T10' part)
```

Example B (full drama: the writes wrap): `(abcddeefffg)` (n = 15)

```
runs: a(-1) b(-1) c(-1) d×6(+4) ee(0) fff(+1) g(-1)    find_start: pos = 3
idx:  0 1 2 3 4 5 6 7 8 9 0 1 2 3 4
      a b c d d d d d e e f f f g    input
      a b c d 6 e 2 f 3 g 1 f f f g  encode from pos: d6 e2 f3 g1
      a b c d 6 e 2 f 3 g 1 a 1 b 1   skipped runs: a1, b1 fill the tail...
      c 1 c d 6 e 2 f 3 g 1 a 1 b 1   ...c1 WRAPS to the front!
      ^^^ ^                           two pieces + a 1-cell hole (the old
'c' at idx 2)
      c 1 d 6 e 2 f 3 g 1 a 1 b 1 .   shift_left by 1: close the hole
      a 1 b 1 c 1 d 6 e 2 f 3 g 1 .   rotate_left by 10 (hole-end 2 +
suffix 8)
answer = a1b1c1d6e2f3g1
```

Note in example B: the stale `(c)` at index 2 *looks* like data but is garbage — we know it by **position**, not by content. (That is also why marking garbage with a special character like `(#)` is a bug: `(#)` might be real input!)

Big picture

```
Level 1: extra array allowed    -> trivial
Level 2: in place, letters only -> 2-pass (shrink forward, grow backward)
Level 3: in place, digits allowed -> find_start + encode + shift + rotate
```

Every tool in Level 3 was already in sections 1-4, and the key insight (`find_start`) is the Gas Station greedy from section 8. **Hard problems are easy problems glued together — learn to see the seams.**